Aprendizaje por refuerzo con verificación certificada como única recompensa: definición formal del entorno `unirl`

Nota técnica del proyecto `rl_metodos`

Carlos Manuel Orrego Franco
Universidad Nacional de Colombia, Sede Manizales

Septiembre de 2026 — borrador v0.2 (con primeros resultados)

1. Propósito

Esta nota fija el lenguaje común de los subproyectos de `rl_metodos`. Todos comparten una misma estructura: un *proponente* produce un objeto de prueba y un *juez* independiente decide si lo acepta. La recompensa proviene únicamente del juez; el proponente nunca se evalúa a sí mismo. Lo que varía entre subproyectos es el juez, y con él el residuo de confianza que la aceptación deja sin certificar. Esa variación es deliberada: es el experimento.

La nota define el proceso de decisión de Markov (MDP) común (§2), la interfaz de verificador y sus tres instancias (§3), las funciones de recompensa (§4), las métricas (§5) y la jerarquía de confianza entre jueces (§6). Los entornos concretos aparecen en la Tabla 1. La versión v0.2 añade los primeros resultados (§8) y corrige el borrador v0.1 donde no coincidía con la implementación; el Apéndice C enumera los cambios y su motivo. Todo lo descrito está implementado en el subproyecto `09_unified` (paquete `unirl`), salvo lo que se marca explícitamente como previsto.

2. El MDP común

Definición 1 (Entorno de demostración verificada)**.** Un entorno es una tupla $\mathcal{E} = (T, \mathcal{A}, V, B)$ donde T es un *objetivo* inmutable (un enunciado y su contexto), $\mathcal{A}$ es un espacio de *propuestas*, V es un *verificador* y $B \in \mathbb{N}$ es un presupuesto de pasos. El MDP asociado es $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ con:

- **Estado** $s_t = (T, \mathcal{C}_t, h_t)$: el objetivo, el *contexto observable* $\mathcal{C}_t$ que el juez devuelve tras el último intento (el paso donde falló la cadena y por qué, las obligaciones abiertas con su texto literal, o el peor caso numérico τ de la propuesta frente a la referencia), y el historial h_t de propuestas y resultados anteriores. En la implementación, s_t se presenta al proponente como un único texto: la descripción del objetivo seguida de cada intento anterior y de su retroalimentación.

- **Acción** $a_t \in \mathcal{A}$: una propuesta completa (una cadena de lemas, una anotación de invariante y variante, o un programa de tamaños de paso). El proponente es un modelo de lenguaje; la acción es su salida tras una extracción determinista (§3.3 describe la de V_I).

- **Transición** $P(s_{t+1} \mid s_t, a_t)$: s_{t+1} incorpora el resultado $V(T, a_t)$ y actualiza $\mathcal{C}_{t+1}$ y h_{t+1}. Es determinista para V_G y, salvo tolerancias numéricas del solver, para V_P. Para V_I no lo es en sentido estricto: Sledgehammer ejecuta varios demostradores en paralelo con un límite de tiempo real, y lo que encuentra puede variar con la carga de la máquina. Por eso las corridas de V_I no se solapan entre sí. Fuera de eso, la estocasticidad vive en el proponente.

- **Recompensa** $R(s_t, a_t) = r(V(T, a_t))$, función únicamente del resultado del juez (§4).
- **Terminación**: el episodio termina en el primer t con $o.\texttt{accepted} = 1$, o cuando $t = B$. Si la llamada al modelo falla tras los reintentos, el paso se registra como PROVIDER_ERROR, sin juez, y consume presupuesto; en las corridas de esta nota no ocurrió.

Observación 1 (El objetivo es inmutable). T no forma parte del espacio de acciones. Una propuesta no puede modificar el enunciado, sus hipótesis ni su tipo, y esto se impone en código, no por convención, de forma distinta en cada juez. En V_G el proponente sólo puede nombrar pasos del grafo; un identificador ajeno es UNKNOWN_LEMMA. En V_P la instancia $(n, \mu, \text{criterio})$ la fija el entorno y la propuesta sólo produce una sucesión de pasos. En V_I el proponente escribe dos huecos del programa (invariante y variante); programa, triple de Hoare y lemas prestados forman una plantilla fija, y toda propuesta que intente salir de su hueco se rechaza con la etiqueta STATEMENT_DRIFT (§4.3). La garantía de fondo la da el kernel: en Isabelle/ITrees la construcción `while_inv_var B I V P` (guarda, invariante, variante y cuerpo) está definida como `iterate B P`, así que las anotaciones no cambian la semántica, y V_I demuestra en el kernel que el programa anotado es igual al programa congelado antes de aceptar . Es la *frontera semántica* del proyecto: el lugar donde la confianza no puede delegarse al juez.

3. La interfaz de verificador y sus tres instancias

Definición 2 (Verificador)**.** Un verificador es una función $V : \mathcal{T} \times \mathcal{A} \to \mathcal{O}$ que devuelve un *resultado*

$$o = (\texttt{accepted}, \texttt{label}, \texttt{partial}, \texttt{cost_s}, \texttt{raw}),$$

con $\texttt{accepted} \in \{0, 1\}$, $\texttt{label} \in \mathcal{L}_V$ y $\texttt{partial} \in [0, 1]$, donde $\mathcal{L}_V$ es un conjunto finito de etiquetas de resultado propio de cada juez, `partial` es una medida de progreso que sólo se usa bajo recompensa moldeada, y `raw` es la salida íntegra del juez para auditoría.

3.1. Juez de grafo de lemas (V_G)

Proviene de los subproyectos 01–05 (su "fase 2"). Un objetivo T es un grafo dirigido acíclico $G = (\Lambda, D)$ de pasos $\lambda \in \Lambda$, cada uno con dependencias $D(\lambda) \subseteq \Lambda$, un subconjunto $\Delta \subset \Lambda$ de *distractores* (pasos falsos o inaplicables, tomados de errores reales de estudiantes) y un paso distinguido QED. Una propuesta es una secuencia $a = (\lambda_1, \ldots, \lambda_k)$. El juez la recorre y acepta si existe $j \le k$ con $\lambda_j = \text{QED}$ tal que, para todo $i \le j$,

$$\lambda_i \notin \Delta, \qquad \lambda_i \notin \{\lambda_1, \ldots, \lambda_{i-1}\}, \qquad D(\lambda_i) \subseteq \{\lambda_1, \ldots, \lambda_{i-1}\};$$

lo que venga después de QED se ignora. Etiquetas, asignadas en el primer paso que falla: ACCEPTED, INVALID_STEP (dependencia no establecida o paso repetido), DISTRACTOR, UNKNOWN_LEMMA, INCOMPLETE (todos los pasos son válidos pero la cadena termina sin llegar a QED). Progreso parcial: si $E(a)$ es el conjunto de pasos establecidos antes del primer fallo y $\text{cl}(\text{QED})$ el cierre de dependencias de QED,

$$\texttt{partial}_G(a) = \frac{|E(a) \cap \text{cl}(\text{QED})|}{|\,\text{cl}(\text{QED})|}.$$

Al proponente se le muestran el enunciado de QED, el texto de cada paso con un identificador anónimo (`L01`, `L02`, ..., barajados con semilla fija) y la longitud mínima de una cadena válida. No se le muestran los títulos de los pasos, porque en 01–05 el título delata al distractor ("Distractor", "Non sequitur"). Tampoco se muestra la refutación que el profesor añadió entre paréntesis al final de 7 de los 36 distractores, por ejemplo "(¡falso: el orden es $\varphi < 2$!)": delata igual que el título. El resto del texto del distractor sí se muestra, porque es el error de estudiante.

3.2. Juez PEP (V_P)

Proviene del subproyecto 06, que se usa sin modificaciones. El objetivo es una instancia: la clase de funciones L-suaves (μ-fuertemente) convexas, un horizonte n y un criterio; una propuesta es un programa `schedule(n)` que genera pasos $(h_0, \ldots, h_{n-1})$. El juez ejecuta el programa, resuelve el problema de estimación de desempeño (PEP) como un programa semidefinido y obtiene el peor caso $\tau(h) = \sup_f f(x_n) - f^*$ con $L = 1$ y $\|x_0 - x^*\| \leq 1$; el certificado dual es la demostración de la cota (Drori and Teboulle, 2014; Taylor et al., 2017). La referencia τ_{ref} es el mínimo entre los óptimos numéricos publicados (Das Gupta et al., 2024), tal como los tabula Grimmer et al. (2024), y lo mejor que encontró la búsqueda propia de 06. Toda mejora afirmada ($\log(\tau_{\text{ref}}/\tau) > 10^{-5}$) se recalcula con tolerancias finas, τ_{fina}, y se certifica en aritmética racional exacta, lo que da una cota $\tau_{\text{cert}} \geq \tau_{\text{fina}}$; sin certificado, no hay aceptación. Etiquetas: CERTIFIED (aceptada), INVALID_SCHEDULE, NOT_IMPROVED, CERTIFICATION_FAILED. La aceptación es

$$\mathbf{1}\big[\log(\tau_{\text{ref}}/\tau_{\text{fina}}) > 10^{-5} \ \wedge \ \text{certificado exacto válido} \ \wedge \ \tau_{\text{cert}} < \tau_{\text{ref}}\big].$$

Para ensayos existe un modo `match` que acepta además $\tau \leq \tau_{\text{ref}}(1+10^{-4})$ con la etiqueta MATCHED; esa aceptación se apoya en el valor numérico del SDP, no en un certificado exacto, y no es el modo por defecto. Progreso parcial: $\text{mín}\{1, \text{máx}\{0, \ 1 + r_P\}\}$, con r_P la recompensa continua de 06 (§4.2).
06 ejecuta el programa propuesto en su propio proceso con los *builtins* restringidos. Dos rondas de revisión adversarial mostraron que así el programa puede alcanzar por introspección los módulos del juez y sustituir la puntuación y el certificado, y que, aun en otro proceso, puede escribir en disco con numpy y subir τ_{ref} en el archivo de referencias. Por eso V_P ejecuta el programa en un intérprete aparte, bajo el sandbox de macOS sin escritura en disco ni red; lee las referencias una sola vez al empezar; puntúa la sucesión literal que devuelve el programa; y al proponente sólo le devuelve el tipo de una excepción, no su texto. Protege el estado del juez; no es un aislamiento completo de la máquina, porque el programa aún puede leer archivos.

3.3. Juez Isabelle (V_I)

Proviene del subproyecto 08, que usa el marco Isabelle/ITrees (Foster et al., 2024) tal como lo emplean Bryant et al. (2026); se usa el controlador de 08 sin modificaciones. Un objetivo es un programa imperativo junto con un triple de Hoare de corrección total $H[P]\,C\,[Q]$, ambos congelados; una propuesta es una anotación $a = (inv, var)$ de invariante y variante de bucle (en una etapa posterior, además una selección de premisas y pruebas Isar para obligaciones abiertas). El proponente responde con las marcas `INVARIANT: <<< ... >>>` y `VARIANT: <<< ... >>>`; la extracción toma el texto entre ellas y, si termina en un símbolo de Isabelle sin cerrar (`\<rceil`), lo cierra, porque el cierre `>>>` puede comerse el `>` final del símbolo. El juez procede en etapas:

0. Comprueba la anotación con una lista blanca de símbolos y demuestra en el kernel que el programa anotado es igual al congelado con anotaciones fijas (`simp only` con `while_inv_var_def`); si no lo es, STATEMENT_DRIFT. La compilación usa pruebas secuenciales y debe terminar sin errores: con pruebas en paralelo, Isabelle sigue ejecutando la teoría aunque una prueba falle, y un test mostró que así un programa alterado pasaba por igual.

1. Inserta la anotación en la plantilla y ejecuta el generador de condiciones de verificación `vcg`.

2. Intenta cerrar cada obligación (VC) por separado con automatización de una línea: `try0` con 5 s y, si falla, Sledgehammer con 60 s. Una sugerencia de Sledgehammer sólo cuenta si su reconstrucción (*preplay*) terminó dentro del tiempo; el controlador de 08, por sí solo, también contaría las que no terminaron.

3. Si todas las VCs tienen una prueba de una línea, ensambla el teorema completo en una teoría nueva (`apply vcg`, un `subgoal` con su prueba por VC, `done`) y lo compila con `isabelle build`.

4. Comprueba que el teorema compilado no dependa de ningún oráculo ni de `skip_proof` (`Thm_Deps.all_oracles`).

Etiquetas: ACCEPTED (todas las etapas pasan: el teorema está en el kernel), VC_OPEN (alguna VC sin prueba de una línea), VCG_FAILED (la anotación no se analiza o no tipa, o `vcg` falla), ASSEMBLY_FAILED (las VCs se cierran por separado pero el teorema ensamblado no compila), UNSOUND (`sorry`, `oops` u oráculos), STATEMENT_DRIFT, TIMEOUT e INVALID_PROPOSAL (falta el invariante o el variante). Progreso parcial: fracción de VCs cerradas en VC_OPEN y ASSEMBLY_FAILED, y 0 en las demás etiquetas. El denominador depende de la anotación (entre 12 y 25 VCs en las corridas de `gpt-6-luna`), así que la fracción no es comparable entre propuestas y, con recompensa moldeada, es manipulable añadiendo conjunciones triviales.

El único objetivo del artículo congelado hasta ahora es `bisection_error_bound`: el programa `bisection` y el triple, con el invariante y el variante vaciados. Se toman del commit `0e7830d` de `isabelle-utp/Verified_Numerical_Methods`. El artículo enlaza otro repositorio, `brydustin/Verification_Of_One_Dim_Optimization`, en el commit `036eea3` (27 de junio de 2026); allí el programa y el triple son idénticos, pero el invariante termina en $(iter = 0 \vee 2(u-l) > tol)$. El invariante de `0e7830d`, que termina en $2(u-l) > tol$, es el que imprime el artículo, y por eso se usa como línea base. Se prestan, copiados literalmente con sus demostraciones, dos lemas de matemática general: `Bolzanos_theorem` y `sign_prop`. No se presta `bisect_root_extract`, un lema de 52 líneas que los autores escribieron para cerrar la postcondición: no es matemática auxiliar sino la prueba manual de dos VCs empaquetada como lema. Tampoco se dan los tres hechos que la demostración del repositorio establece antes de `proof (vcg)` (`a_less_than_b` y dos desigualdades sobre $\log_2((b-a)/tol)$ declaradas `[simp]`). Un segundo objetivo, trivial, sirve de control del juez: una cuenta regresiva. Con la anotación correcta produce ACCEPTED; con un variante que no decrece, VC_OPEN; y una prueba de una línea tramposa (`Skip_Proof.cheat_tac`) queda detectada como oráculo en la etapa 4.

Observación 2 (Existencia frente a descubrimiento). El artículo afirma que `vcg` genera 15 VCs, "13 of which can be proven directly with Isabelle's Sledgehammer"; la columna "Auto." de su Tabla 1 cuenta las VCs cerradas por una llamada directa a un demostrador, es decir, las que tienen una prueba de una línea. No dice con qué límite de tiempo ni en qué máquina. Al reproducir la fila de bisección de su Tabla 1 con sus propios invariantes (§8.2), el conjunto de obligaciones coincide: 15 VCs. Para 13 existe una prueba de una línea que el kernel acepta, pero en esta máquina, con 60 s, la automatización sólo encuentra 9. Las cuatro VCs del decrecimiento del variante se cierran en unos 0,2 s con la única línea `smt` que escribieron los autores (18 hechos, reutilizada para las cuatro con `thus`). Sledgehammer no la redescubre en 60 s sobre las siete premisas que produce `vcg`, ni en 240 s para la primera. En la demostración del repositorio esas VCs se enuncian con una sola premisa, y cabía pensar que adelgazar las premisas bastaba. Se comprobó con `tools/sh_goal.py`: con una sola premisa, y tanto sin como con los tres hechos previos de la demostración, Sledgehammer tampoco la encuentra en 60 s (encontrada: no, sin los hechos; no, con ellos). En esta máquina, adelgazar las premisas no basta: la línea de los autores lleva una selección de 18 hechos que la automatización no reproduce. Las dos VCs restantes (la postcondición) los autores las prueban a mano; en 60 s, Sledgehammer sólo las cierra en una línea cuando se importa `bisect_root_extract`. Por eso V_I distingue tres líneas base: (i) automatización sobre todas las premisas de `vcg`, (ii) automatización con un filtro de relevancia estándar sobre las premisas locales y (iii) selección de premisas como acción explícita del agente. Hoy sólo (i) está implementada; (ii) y (iii) son trabajo futuro. Nunca se mezclan.

4. Funciones de recompensa

4.1. Recompensa base (todos los jueces)

$$r(o) = \begin{cases} 1 & \text{si } o.\texttt{accepted} = 1, \\ 0 & \text{en otro caso.} \end{cases}$$

Es la única recompensa activa por defecto. Es binaria a propósito: cualquier señal intermedia introduce un juicio sobre qué es "progreso", y ese juicio debe declararse, no esconderse.

4.2. Recompensa moldeada (opcional, tras bandera)

$$r_\beta(o) = r(o) + \beta \cdot o.\texttt{partial} \cdot \mathbf{1}[o.\texttt{accepted} = 0], \qquad \beta \in [0, 0.5].$$

Se activa con `--shaped` y `--beta`; el código rechaza β fuera de $[0, 0.5]$. Para V_P se conserva además la recompensa original del subproyecto 06, que es continua por naturaleza: $r_P(o) = \max\{-2,\ \log(\tau_{\text{ref}}/\tau)\}$ si el programa es válido y -3 en otro caso, donde τ es el valor del SDP; si se afirma una mejora, se usa τ_{fina} y, con certificado, τ_{cert}, y sin certificado la recompensa se acota en 0. Se registra en `raw` y define el progreso parcial de V_P. La aceptación de V_P en el modo por defecto es una binarización de r_P con certificado exacto (§3.2), lo que la hace comparable con los otros jueces; la binarización numérica de v0.1, $\mathbf{1}[\tau(h) < \tau_{\text{ref}}]$, no se reporta aparte.

4.3. Penalización por deriva del enunciado

STATEMENT_DRIFT recibe recompensa 0, no negativa, en la configuración base. La razón es metodológica: queremos *contar* cuántas veces el proponente intenta cambiar la pregunta, y una penalización alteraría la frecuencia que se pretende medir. Una variante con penalización $-\delta$ queda disponible tras bandera (`drift_penalty` en la configuración) para experimentos de control.

5. Métricas

Para cada entorno, juez y proponente se registran en JSONL todos los intentos (propuesta, respuesta del modelo, resultado, salida del juez, tiempos) y se calculan:

1. **Intentos hasta aceptación**: mediana y distribución de t^* sobre episodios, con $t^* = B$ si no hay aceptación. Se informa además el número de episodios aceptados, porque la censura en B confunde "aceptado en el último paso" con "no aceptado".

2. **Distribución de etiquetas**: conteo de cada $\ell \in \mathcal{L}_V$ sobre todos los intentos.

3. **Razón de deriva**: para V_I, el cociente entre el número de resultados STATEMENT_DRIFT y el de VC_OPEN; para V_G, el cociente entre DISTRACTOR e INVALID_STEP. Compara cuántas veces el proponente "resuelve" cambiando la pregunta o cayendo en un error de estudiante con cuántas falla de otra forma. En V_I, por construcción (Observación 1), la deriva sólo puede manifestarse como un intento de salir del hueco.

4. **Transferencia**: reducción de t^* en el objetivo T_2 cuando el proponente ha sido entrenado (o condicionado con ejemplos aceptados) sobre T_1, frente a partir de cero. *Prevista; no implementada.*

5. **Comparación entre jueces**: para un mismo método y un mismo proponente, t^* y etiquetas bajo V_G frente a V_I. Es una medida indirecta de cuánto de la "facilidad" de un juez proviene del trabajo humano incorporado en él. Sólo sería directa si ambos jueces juzgaran el mismo enunciado con la misma información, y hoy no es así (§8.5).

6. **Costo**: segundos de juez y de modelo por intento; determina el cuello de botella real.

6. Jerarquía de confianza entre jueces

Los tres jueces aceptan objetos distintos y dejan residuos de confianza distintos. Ordenados de menor a mayor independencia respecto del humano:

juez	qué certifica	en qué reposa la confianza
V_G	validez de una cadena en un grafo dado	el grafo, escrito por el profesor
V_P	una cota numérica de peor caso	la formulación del PEP; para una mejora, el certificado racional exacto; para una igualación (modo `match`), la tolerancia del solver
V_I	un teorema en el kernel de Isabelle/HOL	el kernel; la adecuación del triple congelado; la elección de la matemática importada; el modelo del programa en números reales

Ningún juez elimina la confianza; cada uno la desplaza. V_G la desplaza al autor del grafo. V_P la desplaza a la formulación del PEP y a la implementación del certificador. V_I la desplaza a lugares que el kernel no puede inspeccionar. El primero es la correspondencia entre el triple de Hoare y el enunciado informal que se pretendía probar. El segundo es la elección de la matemática auxiliar: el kernel comprueba las demostraciones de los lemas prestados, pero no decide si un lema "auxiliar" ya contiene la prueba pedida, como `bisect_root_extract`. El tercero es el modelo del programa: el programa verificado opera con números reales exactos, no con aritmética de punto flotante. La automatización (Sledgehammer, los demostradores externos) no está en la base de confianza: sus pruebas se reconstruyen en el kernel, y por eso V_I reensambla y recompila el teorema completo antes de aceptar. La Observación 2 muestra que incluso lo que el artículo de referencia reporta como "automático" contiene decisiones humanas en esa frontera. El objetivo del proyecto es medir ese residuo, no negarlo.

7. Entornos

La Tabla 1 lista los entornos, su juez y el estado de cada uno.

8. Resultados

8.1. Montaje

Qué no se hace. En v0.2 el proponente no aprende de la recompensa: el modelo está congelado, cada episodio empieza sin historial y la única adaptación ocurre dentro del episodio, a través de h_t. La recompensa se registra para los experimentos de entrenamiento y transferencia previstos (§5). Los resultados miden, por tanto, lo que un modelo fijo logra con la retroalimentación de cada juez.

Proponente. Uno solo, en todos los entornos: `gpt-6-luna` de OpenAI con esfuerzo de razonamiento `high`, llamado a través del módulo de proveedores de 06 el 23 de septiembre de 2026. Es una decisión del autor: v0.1 preveía un modelo económico para V_G, y las corridas que se hicieron con `gpt-5.4-mini` quedan archivadas fuera de la nota (`runs/_excluido_solo_luna`). El mensaje de sistema es el mismo para los tres jueces. La API no fija una instantánea del modelo, así que las corridas no son reproducibles bit a bit.

Presupuesto y episodios. $B = 4$ en todos los entornos. V_G: 20 episodios por entorno; la anonimización (semilla 0) es la misma en todos los episodios. V_I: 3 episodios, porque cada intento cuesta minutos de juez.

sub.	método	teorema del grafo	juez	pasos	distr.	cad. mín.
01	bisección	$m_n \to c$, $f(c) = 0$, $\lvert m_n - c\rvert \le (b-a)/2^{n+1}$	V_G; V_I	21	7	14
02	punto fijo	Banach en $[a, b]$: unicidad, convergencia, cotas	V_G; V_I previsto	20	7	13
03	Taylor	resto de Lagrange; convergencia con derivadas acotadas	V_G	20	7	13
04	Newton	convergencia local cuadrática	V_G	18	7	11
05	secante	convergencia local de orden φ	V_G	21	8	13
06	pasos de GD	cota de peor caso $f(x_n) - f^*$	V_P (implementado, sin corridas)	—	—	—
08	GD (suave/PL/cuad.)	convergencia con cota de iteraciones	V_I previsto	—	—	—

Tabla 1: Entornos de `unirl`. "Pasos" cuenta todos los pasos del grafo, incluidos distractores y QED; "cad. mín." es $\lvert \mathrm{cl}(\mathrm{QED})\rvert$. Los objetivos formales de V_I son los teoremas de Bryant et al. (2026); hoy sólo el de bisección (`bisection_error_bound`) está congelado. El enunciado formal no coincide con el del grafo. En bisección, el grafo de V_G concluye que la sucesión infinita m_n converge a una raíz c con $\lvert m_n - c\rvert \le (b-a)/2^{n+1}$, sin tolerancia; V_I exige la terminación de un programa con una raíz a distancia $\le tol$ y exactamente $\lceil \log_2((b-a)/tol)\rceil$ iteraciones. En punto fijo, el grafo de 02 es Banach global en $[a, b]$ y el teorema formal (`fpm_iter_bnd_local_leq`) es local en una bola alrededor del punto fijo. Taylor, Newton y secante sólo disponen de juez de grafo hasta que exista una formalización. v0.1 preveía un proponente económico para V_G y uno de frontera para V_P y V_I; en v0.2 se usa `gpt-6-luna` en todos (§8.1).

Juez Isabelle. MacBook Air (Apple M5, 24 GiB), macOS 26.5.2, Isabelle2025. Sesión del artículo en el commit `0e7830d`; los repositorios de Isabelle/UTP en su último commit hasta el 16 de enero de 2026, inclusive (Apéndice A explica por qué). Compilar desde cero: HOL-Analysis 5 min 9 s (según el registro de la primera compilación; la base de datos del heap guarda 281 s de sesión, sin la escritura del heap), `ITree_Numeric_VCG` 1 min 50 s, sesión del artículo 36 s. Demostradores de Sledgehammer: cvc5, veriT, Z3, E, SPASS, Vampire y Zipperposition, los del paquete de Isabelle2025. Cinco de ellos (SPASS, Vampire, veriT, Z3 y Zipperposition) son binarios x86-64 que corren bajo Rosetta, lo que puede hacer que 60 s rindan menos que en la máquina de los autores. `try0`: 5 s; Sledgehammer: 60 s por VC. Durante todas las corridas un proceso del sistema (`mediaanalysisd`) ocupó unos dos núcleos. El primer intento bajo V_I se solapó con unas corridas de V_G que luego se repitieron; éstas sólo esperan a la red.

Versión del juez. Las corridas de V_I se hicieron antes de añadir la etapa 0 (lista blanca e igualdad de programas en el kernel). A posteriori se pasó esa etapa sobre las 12 propuestas registradas: 0 fallan la lista blanca, 0 dan STATEMENT_DRIFT, y en 12 de 12 el resultado es el que su etiqueta implica (se analiza y es el programa congelado si la etiqueta no es VCG_FAILED; no se analiza si lo es). Ninguna etiqueta publicada cambia (`runs/_baseline/posthoc_stage0.json`).

8.2. Reproducción de la fila de bisección de la Tabla 1 del artículo

Antes de usar V_I como juez se reprodujo la fila de bisección de Bryant et al. (2026) con los invariantes del propio artículo (subproyecto 08). La tabla resume los recuentos; la Observación 2 los interpreta.

criterio	VCs	cerradas en una línea
artículo, Tabla 1	15	13 (87 %)
08: teoría completa del artículo y los 3 hechos previos de su demostración	15	11
sin contar las 2 que usan `bisect_root_extract`	15	9
más la línea `smt` del artículo en las 4 VCs del variante, en el kernel	15	13
V_I de 09 (objetivo congelado: sin `bisect_root_extract` ni esos 3 hechos)	15	9

Con el invariante y el variante del artículo, el juez V_I de 09 da VC_Open con 9 de 15 VCs cerradas: quedan abiertas las 4 del variante y las 2 de la postcondición. Es el mismo recuento que en 08 sin `bisect_root_extract`, así que los tres hechos previos no cambian nada. Este es el listón para el agente: con la anotación de los autores, el juez tampoco acepta.

8.3. Juez de grafo en los cinco métodos

La Tabla 2 resume todas las corridas y la Tabla 3 desglosa las etiquetas de V_G.

método	juez	proponente	ep.	acept.	t^* med.	distribución de t^*	s juez	s LLM
01 bisección	V_G	`gpt-6-luna` (high)	20	20	1	1:20	0,0	8,4
02 punto fijo	V_G	`gpt-6-luna` (high)	20	20	1	1:20	0,0	10,2
03 Taylor	V_G	`gpt-6-luna` (high)	20	20	1	1:16, 2:4	0,0	10,6
04 Newton	V_G	`gpt-6-luna` (high)	20	20	1	1:20	0,0	8,6
05 secante	V_G	`gpt-6-luna` (high)	20	20	1	1:17, 2:3	0,0	24,3
01 bisección	V_I	`gpt-6-luna` (high)	3	0	4	4:3	648,4	72,4

Tabla 2: Resumen por entorno, juez y proponente. t^* es el número de intentos hasta Accepted, con $t^* = B = 4$ si no hubo aceptación; la distribución se da como t^*:episodios. Los segundos son medias por intento.

método	proponente	intentos	Accepted	Invalid_Step	Distractor	razón de deriva
01 bisección	`gpt-6-luna` (high)	20	20	0	0	—
02 punto fijo	`gpt-6-luna` (high)	20	20	0	0	—
03 Taylor	`gpt-6-luna` (high)	24	20	4	0	0/4 = 0,00
04 Newton	`gpt-6-luna` (high)	20	20	0	0	—
05 secante	`gpt-6-luna` (high)	23	20	2	1	1/2 = 0,50

Tabla 3: Etiquetas de V_G sobre todos los intentos. La razón de deriva de V_G es Distractor/Invalid_Step.

Bajo V_G, `gpt-6-luna` acepta todos los episodios de los cinco métodos (20 de 20 en cada uno; Tabla 2), casi siempre al primer intento: t^* mediano 1 en los cinco. Los pocos fallos son casi todos de orden (Invalid_Step): el modelo usa un paso antes de establecer sus dependencias y corrige con la retroalimentación. Sólo en secante cae una vez en un distractor (1 en 23 intentos). Con el texto de los pasos a la vista y la longitud mínima anunciada, V_G mide sobre todo si el modelo respeta dependencias explícitas, no si sabe demostrar.

8.4. Bisección bajo V_G y bajo V_I

método	proponente	intentos	VC_Open	razón de deriva
01 bisección	`gpt-6-luna` (high)	12	12	0/12 = 0,00

Tabla 4: Etiquetas de V_I sobre todos los intentos. La razón de deriva de V_I es Statement_Drift/VC_Open.

Tabla 5: Intentos de V_I en orden. "Cerradas" cuenta las VCs con prueba de una línea; el tipo de cada VC abierta se asigna por su conclusión y sus premisas (heurística). Las anotaciones están en el Apéndice B.

proponente	ep.	paso	resultado	cerradas	VCs abiertas (tipo)	s juez
`gpt-6-luna` (high)	1	1	VC_Open	5/12	invariante 4, postcondición 1, variante 2	599
`gpt-6-luna` (high)	1	2	VC_Open	7/14	invariante 4, postcondición 1, variante 2	643
`gpt-6-luna` (high)	1	3	VC_Open	15/25	invariante 4, postcondición 2, variante 4	729
`gpt-6-luna` (high)	1	4	VC_Open	11/21	invariante 4, postcondición 2, variante 4	731
`gpt-6-luna` (high)	2	1	VC_Open	6/13	invariante 4, postcondición 1, variante 2	586
`gpt-6-luna` (high)	2	2	VC_Open	7/14	invariante 4, postcondición 1, variante 2	604
`gpt-6-luna` (high)	2	3	VC_Open	8/16	invariante 6, postcondición 2	707
`gpt-6-luna` (high)	2	4	VC_Open	9/16	invariante 4, postcondición 1, variante 2	553
`gpt-6-luna` (high)	3	1	VC_Open	7/17	invariante 4, postcondición 2, variante 4	743
`gpt-6-luna` (high)	3	2	VC_Open	8/16	invariante 4, postcondición 4	571
`gpt-6-luna` (high)	3	3	VC_Open	9/19	invariante 4, postcondición 2, variante 4	663
`gpt-6-luna` (high)	3	4	VC_Open	9/19	invariante 4, postcondición 2, variante 4	652

Con el mismo proponente y el mismo presupuesto, los dos jueces dan resultados opuestos. Bajo V_G, `gpt-6-luna` acepta 20 de 20 episodios con t^* mediano 1; bajo V_I, 0 de 3, con sus 12 intentos en VC_Open. La mejor propuesta cerró 15/25 VCs y la media fue 0,49, frente a 9/15 con la anotación del artículo. Cada intento de V_I costó en promedio 648 s de juez, frente a 72 s de modelo.

Tres rasgos de las propuestas explican dónde se atasca. Primero, 6 de 12 meten la existencia de la raíz en el invariante ($\exists c \in [l, u].\ f(c) = 0$), como en una demostración de libro. Eso traslada el argumento de Bolzano a la preservación del invariante, y en las 6 esa VC queda abierta: Sledgehammer no la cierra en 60 s. El artículo hace lo contrario: deja la raíz fuera del invariante y la extrae al final con un lema escrito a mano. Segundo, 6 usan un variante en función del intervalo, $\lceil \log_2((u-l)/tol) \rceil$, y 6 uno en función de `iter`, como el del artículo; en ambos casos las VCs del variante que quedan abiertas (30 de 30) son aritmética con $\lceil \log_2 \cdot \rceil$, la misma que los autores resuelven con su línea `smt` de 18 hechos. Tercero, en los 12 intentos con VCs quedó abierta la postcondición: sin `bisect_root_extract`, ninguna anotación consigue que la automatización extraiga la raíz. Sumando sobre los intentos, las VCs abiertas fueron 50 de invariante, 30 de variante y 21 de postcondición (clasificación heurística por la conclusión y las premisas de cada VC). Ningún invariante coincide con el del artículo (0 de 12); el variante sí, literalmente, en 3, y es la elección natural (Apéndice B).

La diferencia entre jueces no dice que V_I sea "más difícil" en abstracto. Dice dónde está el trabajo: en V_G el profesor ya eligió los lemas, cerró la aritmética y escribió el argumento de Bolzano, y el agente sólo los ordena; en V_I esas tres cosas siguen abiertas, y la automatización de una línea no las hace sola ni con la anotación de los autores. Para que V_I sea un entorno de aprendizaje útil hace falta darle al agente más acciones que la anotación: selección de premisas, lemas auxiliares propuestos por él y verificados, o pruebas Isar (experimento 5).

8.5. Amenazas a la validez

- **Muestras pequeñas y un solo proponente.** Tres episodios en V_I bastan para describir, no para estimar tasas, y con un solo modelo no se puede separar el efecto del juez del efecto del modelo.
- **Enunciados distintos.** V_G y V_I no juzgan el mismo teorema de la bisección (Tabla 1); la comparación mide dos formas de "demostrar la bisección", no dos jueces sobre un mismo enunciado.
- **Lo que ve el proponente en V_G.** Los textos de los pasos revelan buena parte de sus dependencias, y el mensaje dice la longitud mínima de la cadena; la tarea se parece más a ordenar que a demostrar. La semilla de anonimización es la misma en los 20 episodios.
- **Contaminación.** El invariante y el variante del artículo son públicos (arXiv y GitHub desde 2025). Un modelo de frontera puede recordarlos, y un acierto bajo V_I podría ser recuerdo y

no descubrimiento; las propuestas se comparan con las del artículo en el Apéndice B.

- **No determinismo de V_I.** Sledgehammer depende del tiempo real y de la carga; cinco demostradores corren bajo Rosetta y un proceso del sistema ocupó dos núcleos. Otra máquina puede cerrar VCs distintas con los mismos 60 s.
- **Versiones y defectos corregidos.** Las versiones de Isabelle/UTP no son las que fija CyPhyAssure, y los modelos de la API no tienen instantánea fija. Se descartaron y repitieron las corridas de V_G porque las refutaciones del profesor eran visibles en 7 distractores, un defecto del entorno y no del proponente; quedan archivadas con su explicación. Un defecto del formato de respuesta de V_I (el cierre `>>>` choca con `\<rceil>`) sólo afectó a corridas con `gpt-5.4-mini`, hoy excluidas; en las de `gpt-6-luna` no cambia ninguna propuesta.

9. Experimentos previstos

1. *Hecho* (§8.3). Bisección bajo V_G, 20 episodios: primera medición de t^* y de la razón de deriva, extendida a los cinco métodos. Con `gpt-6-luna` en lugar del proponente económico previsto.
2. *Hecho* (§8.4). Bisección bajo V_I con el mismo proponente y el mismo presupuesto $B = 4$.
3. Punto fijo bajo ambos jueces; transferencia desde bisección. Requiere congelar el teorema `fpm_iter_bnd_local_leq` del artículo.
4. Descenso de gradiente: puente entre V_P y V_I. Para el paso constante $1/L$, comparar la cota que calcula V_P (numérica; certificada sólo si mejora τ_{ref}) con la cota de iteraciones que garantiza el teorema formal. Hay que armonizar antes las hipótesis: el teorema formal es para funciones L-suaves no necesariamente convexas, mide $\|\nabla f\|$ y parte de $f(x_0) - f^*$; el PEP de 06 es convexo y parte de $\|x_0 - x^*\|$.
5. *Nuevo.* Líneas base (ii) y (iii) de la Observación 2 para V_I, y pruebas Isar para las VCs que la automatización deja abiertas.

A. Reproducción

```
cd 09_unified && source .venv/bin/activate
sh logs/run_graph_luna.sh            # V_G con gpt-6-luna en 02-05, en paralelo
python -m unirl run --config configs/bisection_graph.toml --episodes 20      # V_G biseccion
python -m unirl run --config configs/bisection_isabelle.toml --episodes 3    # V_I (en serie)
python tools/posthoc_stage0.py       # etapa 0 sobre las propuestas registradas
python tools/baseline_paper.py       # invariante del articulo bajo V_I
python tools/sh_goal.py --timeout 60 # VC del variante con 7 premisas y con 1
python -m unirl summary runs/
python -m unirl tables runs/ --out docs/nota
cd docs/nota && latexmk -pdf unirl_definicion.tex
```

Cada corrida guarda el archivo `attempts.jsonl` en la carpeta `runs/<entorno>/<fecha>-<modelo>-<esfuerzo>/`, junto a un `meta.json` con la configuración, el objetivo presentado (con su hash) y la huella del código de `unirl`. Las carpetas que empiezan por "_" son pilotos, corridas descartadas (con un `LEEME.md`) y líneas base; no entran en las tablas.

Versiones. Isabelle2025; artículo en `isabelle-utp/Verified_Numerical_Methods@0e7830d`; Isabelle/UTP en el meta-repositorio `CyPhyAssure@a41662b`, con los submódulos `interaction-trees`, `Abstract_Prog_Syntax`, `Optics` y `Z_Toolkit` movidos a su último commit hasta el 16 de enero de 2026 (`d3bc22c`, `b97410d`, `6c3e513`, `a081e1e`); el resto, incluido el espejo del AFP 2025, en el commit que fija CyPhyAssure. Motivos: con los submódulos de CyPhyAssure, `ITree_VCG` no

compila (su `interaction-trees` es anterior a la sintaxis $H[P]\,C\,[Q]$ y usa una constante que ya no existe); con los commits del 23 de julio de 2026 falla `ITree_CSP` por el cambio a la sintaxis IsaCircus; con los del 16 de enero compila todo y los cuatro teoremas del artículo pasan sin avisos ni `sorry`. El repositorio del artículo declara `root.tex` sin versionarlo; se crearon dos mínimos. La lista completa está en `~/isabelle/PINS.md`.

B. Anotaciones propuestas por el agente bajo V_I

Anotación del artículo (línea base):

```
INVARIANT: fl = f(l) \<and> fu = f(u) \<and> (l = xmid \<or> u = xmid) \<and> a \<le> l \<and>
    u \<le> b
    \<and> l < u \<and> fl * fu \<le> 0 \<and> u - l = (b - a) / 2^iter \<and> (2 * (u - l) >
    tol)
VARIANT:   nat(\<lceil>log 2 ((b - a) / tol)\<rceil>) - iter
```

Las del agente, tal como quedaron tras la extracción:

`gpt-6-luna` (high), episodio 1, paso 1: VC_Open

```
INVARIANT: a \<le> l \<and> l < u \<and> u \<le> b \<and> fl = f l \<and> fu = f u \<and> fl
    * fu \<le> 0 \<and> u-l = (b-a)/2^iter \<and> iter + nat \<lceil>log 2
    ((u-l)/tol)\<rceil> = nat \<lceil>log 2 ((b-a)/tol)\<rceil> \<and> (\<exists>c::real.
    f(c) = 0 \<and> a < c \<and> c < b \<and> l \<le> c \<and> c \<le> u \<and>
    \<bar>c-xmid\<bar> \<le> (b-a)/2^iter)
VARIANT:   nat \<lceil>log 2 ((u-l)/tol)\<rceil>
```

`gpt-6-luna` (high), episodio 1, paso 2: VC_Open

```
INVARIANT: a \<le> l \<and> l < u \<and> u \<le> b \<and> fl = f l \<and> fu = f u \<and> fl
    * fu \<le> 0 \<and> u - l = (b-a)/2^iter \<and> tol/2 < u-l \<and> iter + nat
    \<lceil>log 2 ((u-l)/tol)\<rceil> = nat \<lceil>log 2 ((b-a)/tol)\<rceil> \<and>
    (\<exists>c::real. f(c) = 0 \<and> a < c \<and> c < b \<and> l \<le> c \<and> c \<le> u
    \<and> \<bar>c-xmid\<bar> \<le> (b-a)/2^iter)
VARIANT:   nat \<lceil>log 2 ((b-a)/tol)\<rceil> - iter
```

`gpt-6-luna` (high), episodio 1, paso 3: VC_Open

```
INVARIANT: a \<le> l \<and> l < u \<and> u \<le> b \<and> fl = f l \<and> fu = f u \<and> fl
    * fu \<le> 0 \<and> u - l = (b-a)/2^iter \<and> tol/2 < u-l \<and> (xmid = l \<or> xmid
    = u) \<and> f a \<noteq> 0 \<and> f b \<noteq> 0 \<and> log 2 ((u-l)/tol) = log 2
    ((b-a)/tol) - real iter \<and> int iter + \<lceil>log 2 ((b-a)/tol) - real iter\<rceil>
    = \<lceil>log 2 ((b-a)/tol)\<rceil>
VARIANT:   nat (\<lceil>log 2 ((b-a)/tol) - real iter\<rceil>)
```

`gpt-6-luna` (high), episodio 1, paso 4: VC_Open

```
INVARIANT: a \<le> l \<and> l < u \<and> u \<le> b \<and> fl = f l \<and> fu = f u \<and> fl
    * fu \<le> 0 \<and> f a \<noteq> 0 \<and> f b \<noteq> 0 \<and> u - l = (b-a)/2^iter
    \<and> tol/2 < u-l \<and> (xmid = l \<or> xmid = u) \<and> iter + nat \<lceil>log 2
    ((u-l)/tol)\<rceil> = nat \<lceil>log 2 ((b-a)/tol)\<rceil>
VARIANT:   nat \<lceil>log 2 ((u-l)/tol)\<rceil>
```

`gpt-6-luna` (high), episodio 2, paso 1: VC_Open

```
INVARIANT: a \<le> l \<and> l \<le> u \<and> u \<le> b \<and> fl = f l \<and> fu = f u
    \<and> fl * fu \<le> 0 \<and> (\<exists>c::real. f c = 0 \<and> c \<in> {a<..<b} \<and>
    c \<in> {l..u}) \<and> xmid \<in> {l..u} \<and> u-l = (b-a)/2^iter \<and> iter + nat
    \<lceil>log 2 ((u-l)/tol)\<rceil> = nat \<lceil>log 2 ((b-a)/tol)\<rceil>
VARIANT:   nat \<lceil>log 2 ((u-l)/tol)\<rceil>
```

`gpt-6-luna` (high), episodio 2, paso 2: VC_Open

```
INVARIANT: continuous_on {a..b} f \<and> a \<le> l \<and> l \<le> u \<and> u \<le> b \<and>
    fl = f l \<and> fu = f u \<and> fl * fu \<le> 0 \<and> (\<exists>c::real. f c = 0 \<and>
    c \<in> {a<..<b} \<and> c \<in> {l..u}) \<and> xmid \<in> {l..u} \<and> u-l =
    (b-a)/2^iter \<and> iter + nat \<lceil>log 2 ((u-l)/tol)\<rceil> = nat \<lceil>log 2
    ((b-a)/tol)\<rceil>
VARIANT:   nat \<lceil>log 2 ((b-a)/tol)\<rceil> - iter
```

gpt-6-luna (high), episodio 2, paso 3: VC_Open

```
INVARIANT: continuous_on {a..b} f \<and> a \<le> l \<and> l \<le> u \<and> u \<le> b \<and>
    fl = f l \<and> fu = f u \<and> fl * fu \<le> 0 \<and> (\<exists>c::real. f c = 0 \<and>
    c \<in> {a<..<b} \<and> c \<in> {l..u}) \<and> xmid \<in> {l..u} \<and> u-l =
    (b-a)/2^iter \<and> iter + nat \<lceil>log 2 ((u-l)/tol)\<rceil> = nat \<lceil>log 2
    ((b-a)/tol)\<rceil> \<and> iter \<le> nat \<lceil>log 2 ((b-a)/tol)\<rceil> \<and> (tol
    < u-l \<longrightarrow> iter < nat \<lceil>log 2 ((b-a)/tol)\<rceil>)
VARIANT:   nat \<lceil>log 2 ((b-a)/tol)\<rceil> - iter
```

gpt-6-luna (high), episodio 2, paso 4: VC_Open

```
INVARIANT: continuous_on {a..b} f \<and> a \<le> l \<and> l \<le> u \<and> u \<le> b \<and>
    fl = f l \<and> fu = f u \<and> fl * fu \<le> 0 \<and> (\<exists>c::real. f c = 0 \<and>
    c \<in> {a<..<b} \<and> c \<in> {l..u}) \<and> xmid \<in> {l..u} \<and> u-l =
    (b-a)/2^iter \<and> tol/2 < u-l \<and> int iter + \<lceil>log 2 ((u-l)/tol)\<rceil> =
    \<lceil>log 2 ((b-a)/tol)\<rceil>
VARIANT:   nat \<lceil>log 2 ((u-l)/tol)\<rceil>
```

gpt-6-luna (high), episodio 3, paso 1: VC_Open

```
INVARIANT: a \<le> l \<and> l \<le> u \<and> u \<le> b \<and> fl = f l \<and> fu = f u
    \<and> fl * fu \<le> 0 \<and> (xmid = l \<or> xmid = u) \<and> u-l = (b-a) / 2^iter
    \<and> int iter \<le> \<lceil>log 2 ((b-a)/tol)\<rceil>
VARIANT:   nat (\<lceil>log 2 ((b-a)/tol)\<rceil> - int iter)
```

gpt-6-luna (high), episodio 3, paso 2: VC_Open

```
INVARIANT: a \<le> l \<and> l \<le> u \<and> u \<le> b \<and> fl = f l \<and> fu = f u
    \<and> fl * fu \<le> 0 \<and> (xmid = l \<or> xmid = u) \<and> u-l = (b-a) / 2^iter
    \<and> int iter \<le> \<lceil>log 2 ((b-a)/tol)\<rceil> \<and> (tol < u-l
    \<longrightarrow> int iter < \<lceil>log 2 ((b-a)/tol)\<rceil>)
VARIANT:   nat (\<lceil>log 2 ((b-a)/tol)\<rceil> - int iter)
```

gpt-6-luna (high), episodio 3, paso 3: VC_Open

```
INVARIANT: a \<le> l \<and> l \<le> u \<and> u \<le> b \<and> fl = f l \<and> fu = f u
    \<and> fl * fu \<le> 0 \<and> (xmid = l \<or> xmid = u) \<and> u-l = (b-a) / 2^iter
    \<and> tol/2 < u-l \<and> int iter + \<lceil>log 2 ((u-l)/tol)\<rceil> = \<lceil>log 2
    ((b-a)/tol)\<rceil>
VARIANT:   nat (\<lceil>log 2 ((u-l)/tol)\<rceil>)
```

gpt-6-luna (high), episodio 3, paso 4: VC_Open

```
INVARIANT: a \<le> l \<and> l \<le> u \<and> u \<le> b \<and> fl = f l \<and> fu = f u
    \<and> fl * fu \<le> 0 \<and> (xmid = l \<or> xmid = u) \<and> u-l = (b-a) / 2^iter
    \<and> tol/2 < u-l \<and> int iter + \<lceil>log 2 ((u-l)/tol)\<rceil> = \<lceil>log 2
    ((b-a)/tol)\<rceil>
VARIANT:   nat (\<lceil>log 2 ((u-l)/tol)\<rceil>)
```

C. Cambios respecto al borrador v0.1

1. **Transición (§2).** v0.1 la declaraba determinista para todos los jueces; V_I no lo es en sentido estricto. Se documenta PROVIDER_ERROR.

2. **Inmutabilidad (Observación 1).** STATEMENT_DRIFT es una etiqueta de V_I; en V_G y V_P la inmutabilidad se impone de otro modo. Se añade el hecho semántico `while_inv_var B I V P = iterate B P`.

3. **V_G.** INCOMPLETE es un resultado por intento (la cadena no llega a QED), no por presupuesto; INVALID_STEP incluye el paso repetido; los pasos posteriores a QED se ignoran. Se documenta qué ve el proponente.

4. **V_P.** La aceptación por defecto es mejora estricta con certificado exacto, como pedía v0.1; se escribe su condición exacta. El programa propuesto se ejecuta en un intérprete aparte y sin escritura en disco, y las referencias se fijan al empezar, porque la revisión adversarial mostró dos formas de falsear un certificado. El modo `match`, añadido para ensayos, no es un certificado exacto. La binarización numérica $\mathbf{1}[\tau(h) < \tau_{\text{ref}}]$ ya no se reporta aparte, y r_P se escribe como la calcula 06.

5. **V_I.** v0.1 pedía "teorema en el heap" sin decir cómo se construye; ahora ACCEPTED exige reensamblar el teorema completo con las pruebas de una línea en una teoría nueva, compilarlo y comprobar que no haya oráculos. A diferencia del controlador de 08, una sugerencia de Sledgehammer sin *preplay* terminado no cuenta. Se añaden las etiquetas VCG_FAILED, ASSEMBLY_FAILED e INVALID_PROPOSAL. Tras una revisión adversarial del juez, la deriva deja de depender de una expresión regular: la anotación pasa una lista blanca y el kernel demuestra que el programa anotado es el congelado (etapa 0), y el teorema aceptado se enuncia sobre el programa congelado. Las corridas de V_I son anteriores a este cambio; la comprobación a posteriori de la §8.1 muestra que ninguna etiqueta publicada cambia.UNSOUND ya no menciona "axiomas adicionales": la plantilla fija impide añadirlos, y la etapa 4 detecta oráculos, no axiomas. Se documentan la extracción y la procedencia del objetivo.

6. **Observación 2.** v0.1 decía que "las cuatro restantes requieren adelgazar las premisas o dependen de un lema de 40 líneas". Son dos grupos distintos: las cuatro del variante tienen una prueba de una línea que Sledgehammer no redescubre, y las dos de la postcondición dependen del lema (52 líneas). El efecto de adelgazar las premisas se comprobó con `tools/sh_goal.py`. v0.1 presentaba las líneas base (ii) y (iii) como existentes; son trabajo futuro.

7. **Confianza (§6).** Se añaden el modelo del programa en números reales y la no confianza en la automatización; "la matemática importada" pasa a "la elección de la matemática importada"; para V_P se distingue mejora (certificado exacto) de igualación (tolerancia del solver).

8. **Métricas.** Se añade el número de episodios aceptados; la comparación entre jueces pasa de "medida directa" a "medida indirecta"; la transferencia queda marcada como prevista; el costo incluye el tiempo del modelo.

9. **Entornos (Tabla 1).** Se marca qué está implementado, se corrige la cota del grafo de bisección a $(b-a)/2^{n+1}$ y se advierte que los enunciados formales difieren de los del grafo.

10. **Experimentos y proponente.** Los experimentos 1 y 2 están hechos, con un solo proponente de frontera (`gpt-6-luna`) en lugar del económico previsto para V_G, por decisión del autor. Se añade el experimento 5.

11. **Forma.** El conjunto de etiquetas pasa a $\mathcal{L}_V$ para no confundirlo con la constante L; $\nVdash$ pasa a $\mathbf{1}$; se añaden la cita del marco Isabelle/ITrees y una sección de montaje y amenazas a la validez.

Referencias

Bryant, D., Huerta y Munive, J. J., and Foster, S. (2026). Verifying numerical methods with Isabelle/HOL. v3, 21 Sep 2026; código: github.com/isabelle-utp/Verified_Numerical_Methods.

Das Gupta, S., Van Parys, B. P. G., and Ryu, E. K. (2024). Branch-and-bound performance estimation programming: A unified methodology for constructing optimal optimization methods. Mathematical Programming.

Drori, Y. and Teboulle, M. (2014). Performance of first-order methods for smooth convex minimization: A novel approach. *Mathematical Programming*, 145(1–2):451–482.

Foster, S., Hur, C.-K., and Woodcock, J. (2024). Unifying model execution and deductive verification with interaction trees in Isabelle/HOL. *ACM Transactions on Software Engineering and Methodology*.

Grimmer, B., Shu, K., and Wang, A. L. (2024). Composing optimized stepsize schedules for gradient descent.

Taylor, A. B., Hendrickx, J. M., and Glineur, F. (2017). Smooth strongly convex interpolation and exact worst-case performance of first-order methods. *Mathematical Programming*, 161(1–2):307–345.